

Fast and Accurate Domain Adaptation for Irregular and Regular Tensor Decomposition

Junghun Kim, Ka Hyun Park, Jun-Gi Jang, and U Kang

Abstract—Many real-world datasets including stock prices or disease records are represented as regular or irregular tensors across multiple domains. *How can we accurately capture patterns from both irregular and regular tensors in a newly emerging domain by leveraging existing ones from multiple domains?* This problem is crucial for applications such as identifying patterns of new diseases using data from existing ones. A main challenge is that the new target tensors contain limited information due to their recent emergence. Previously, PARAFAC2- and PARAFAC-based methods have been widely used to find patterns in irregular and regular tensors, respectively, through decomposing them into latent factors. However, they cannot effectively transfer knowledge from previously known tensors to the new one.

In this work, we propose a fast and accurate domain adaptation method for tensor decomposition. We propose META-P2 for irregular tensors and META-P for regular tensors. Both META-P2 and META-P learn general and easily-adaptable information—referred to as the meta factor—from multiple source domains. Using this meta factor, they efficiently identify patterns in a new target tensor. Extensive experiments on real-world datasets show that META-P2 and META-P achieve the state-of-the-art performance across various downstream tasks, including missing value prediction and anomaly detection.

Index Terms—domain adaptation, meta-learning, irregular tensor, regular tensor, tensor decomposition.

I. INTRODUCTION

How can we quickly and accurately capture patterns from both irregular and regular tensors in a newly emerging domain, leveraging existing tensors across multiple domains? Many real-world datasets are represented as irregular tensors across multiple domains. For instance, stock data from diverse industries are modeled as irregular tensors across multiple domains. Although listed at different times, these tensors exhibit common features, such as opening and closing prices. A regular tensor can be derived from an irregular tensor by imposing a single condition: the varying time lengths are made consistent. For example, restricting the period to the past month ensures that the time axes for all stocks have the same length, resulting in regular tensors across multiple domains.

In these stock scenarios, the objective is to efficiently and accurately capture patterns of stocks in a newly emerged industry by leveraging those from pre-existing ones (see Fig. 1). This problem extends to various other contexts, such

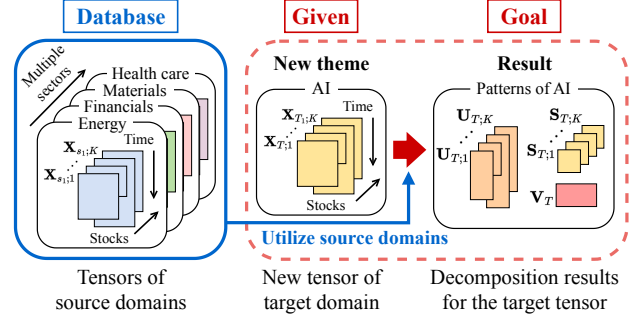


Fig. 1. An example of domain adaptation for irregular tensors. We are given a stock database represented as irregular tensors across multiple domains. Each domain corresponds to a specific sector, and each slice $X_{d:k}$ contains time-varying stock information (e.g., opening prices) for the k -th stock in sector d . By limiting the stock data to a specific time period, the tensors become regular tensors. We then encounter a new unseen themed-stocks (target tensor). The goal is to discover patterns in this new tensor through tensor decomposition, leveraging data from multiple source domains.

as identifying new disease patterns using pre-existing disease data or finding behavioral patterns of a new individual based on the records from multiple people.

Tensor decomposition methods find patterns of data by representing them with low-rank factor matrices [1]–[4]. Among them, PARAFAC2 [5] is widely used for analyzing irregular tensors in various applications, such as missing value prediction [6], anomaly detection [7], failure prediction [8], and image recommendation [9]. For regular tensors, PARAFAC [10] is commonly employed: community detection [11], missing value prediction [12], and anomaly detection [13].

However, those methods cannot be directly applied in our problem. They present two key challenges: 1) the target tensor contains limited information due to its recent emergence, and 2) disparities between domains (e.g., stock movements vary across industries) complicate the transfer of patterns. A naive approach is to perform decomposition solely on the target tensor, but this strategy overlooks valuable information from source domains, leading to inaccurate results. Another option is to decompose both source and target tensors together by combining them. However, this approach fails to find accurate latent factors of the target tensor if the source and target exhibit different properties. Additionally, it requires recomputing the decomposition for all domains whenever a new target tensor emerges, leading to substantial computational inefficiency. Thus, it is crucial to selectively extract useful information that helps analyze the target tensor in advance.

In this work, we propose a fast and accurate domain adaptation method for tensor decomposition. Specifically, we propose META-P2 for handling irregular tensors and extend it to

Junghun Kim (bandalg97@snu.ac.kr) is with Interdisciplinary Program in Artificial Intelligence, Seoul National University, South Korea.

Ka Hyun Park (kahyun.park@snu.ac.kr) is with Department of Computer Science, Seoul National University, South Korea.

Jun-Gi Jang (junggi@dgist.ac.kr) is with Department of Electrical Engineering and Computer Science, DGIST, South Korea.

U Kang (ukang@snu.ac.kr, corresponding author) is with Department of Computer Science, Seoul National University, South Korea.

META-P for regular tensors. META-P2 and META-P carefully learn a general latent factor from multiple source domains based on the meta-learning approach (see Section II-D for details). The latent factor matrix is then used for decomposing a new target tensor. This approach facilitates the effective transfer of general knowledge from source domains to the target domain, ensuring accurate decomposition of the target.

META-P2 and META-P effectively address the limitations of previous approaches. First, they efficiently supplement the limited information of the target tensor by pre-extracting useful information from the source tensors. Unlike existing methods, META-P2 and META-P eliminate the need for joint tensor decomposition across all domains whenever a new target domain emerges. Second, META-P2 and META-P accurately decompose the target tensor, even when the source and target domains differ significantly. This is achieved by extracting *general* and *easily adaptable* information from the source domains, rather than domain-specific knowledge, through meta-learning-based tensor decomposition.

Our contributions are summarized as follows:

- **Method.** We propose META-P2 and META-P, fast and accurate domain adaptation methods for decomposing irregular and regular tensors, respectively. They effectively find the patterns of a new target tensor via meta-learning.
- **Theory.** We theoretically analyze META-P2 and META-P, focusing on its properties such as time complexities and the connection between their meta-update step and the update step in previous meta-learning approaches.
- **Experiments.** Extensive experiments show that META-P2 and META-P achieve the state-of-the-art performance on real-world downstream tasks including missing value prediction and anomaly detection.

The code and datasets are available at <https://github.com/snudatalab/Meta-P2>.

II. PRELIMINARIES AND PROBLEM

A. Tensor Notations and Operations

We provide the symbols used in this paper in Supplementary Material (Section A).

Irregular Tensor. We denote an irregular tensor as a set $\{\mathbf{X}_k\}_{k=1}^K$ where $\mathbf{X}_k \in \mathbb{R}^{I_k \times J}$ is the k -th slice of the tensor, and K is the number of slices. Note that the row sizes I_k of the slices are different for all k . An irregular tensor in domain d is denoted as $\{\mathbf{X}_{d;k}\}_{k=1}^K$. We denote $\{\mathbf{X}_{d;k}\}_{k=1}^K \in \mathbb{R}^{I_k \times J \times K}$ as $\{\mathbf{X}_d\}$ in this paper for brevity.

Regular Tensor. Regular tensors are a specific form of irregular tensors, with the added constraint that all dimensions are of equal length. For consistency of expression, we denote a regular tensor as a set $\{\mathbf{X}_k\}_{k=1}^K$ where the slices $\mathbf{X}_k$ for all k have the same shape ($\in \mathbb{R}^{I \times J}$). A regular tensor in domain d is represented by $\{\mathbf{X}_{d;k}\}_{k=1}^K$, which we denote as $\{\mathbf{X}_d\}$.

Mode- n Matricization. Given a tensor $\mathbf{X} \in \mathbb{R}^{I_1 \times \dots \times I_N}$, this operation arranges the mode- n fibers to be the columns of the resulting matrix; the result is of size $(I_n, \prod_{m=1, m \neq n}^N I_m)$. We denote mode- n matricization of $\mathbf{X}$ as $\mathbf{X}_{(n)}$.

Khatri-Rao Product. Given two matrices $\mathbf{A} \in \mathbb{R}^{p \times r}$ and $\mathbf{B} \in \mathbb{R}^{q \times r}$, the Khatri-Rao product of them is denoted

by $\mathbf{A} \odot \mathbf{B} \in \mathbb{R}^{pq \times r}$. The product $\mathbf{A} \odot \mathbf{B}$ is computed as $[\mathbf{A}(:, 1) \otimes \mathbf{B}(:, 1) | \dots | \mathbf{A}(:, r) \otimes \mathbf{B}(:, r)]$ where $\otimes$ is Kronecker product and $|$ is the horizontal concatenation. The product $\mathbf{A}(:, k) \otimes \mathbf{B}(:, k)$ of k -th column is computed as $[\mathbf{A}(1, k)\mathbf{B}(:, k)^\top | \dots | \mathbf{A}(p, k)\mathbf{B}(:, k)^\top]^\top$.

B. PARAFAC2 Decomposition

PARAFAC2 analyzes an irregular tensor by approximating it with a multiplication of latent factor matrices. The objective function of PARAFAC2 is as follows:

$$\min_{\{\mathbf{U}_k\}, \{\mathbf{S}_k\}, \mathbf{V}} \sum_{k=1}^K \|\mathbf{X}_k - \mathbf{U}_k \mathbf{S}_k \mathbf{V}^\top\|_F^2 \quad (1)$$

where $\mathbf{X}_k \in \mathbb{R}^{I_k \times J}$ is a k -th slice of the original irregular tensor $\{\mathbf{X}_k\}_{k=1}^K \in \mathbb{R}^{I_k \times J \times K}$, and $\mathbf{U}_k \in \mathbb{R}^{I_k \times R}$, $\mathbf{S}_k \in \mathbb{R}^{R \times R}$, and $\mathbf{V} \in \mathbb{R}^{J \times R}$ are the decomposed factor matrices of $\mathbf{X}_k$ with rank R . Note that $\mathbf{V}$ is shared along the slices.

To ensure the uniqueness of the results, [5] replaced $\mathbf{U}_k$ with $\mathbf{Q}_k \mathbf{H}$ where $\mathbf{Q}_k \in \mathbb{R}^{I_k \times R}$ is a column orthogonal matrix, and $\mathbf{H} \in \mathbb{R}^{R \times R}$ is a common matrix for all the slices. Then the objective function in Equation (1) is reformulated as

$$\min_{\{\mathbf{Q}_k\}, \{\mathbf{S}_k\}, \mathbf{H}, \mathbf{V}} \sum_{k=1}^K \|\mathbf{X}_k - \mathbf{Q}_k \mathbf{H} \mathbf{S}_k \mathbf{V}^\top\|_F^2. \quad (2)$$

ALS (Alternating Least Square) is commonly used for optimizing Equation (2) due to its fast speed and accuracy [14]. CP-ALS, which uses ALS for PARAFAC2 decomposition, iteratively updates each factor matrix while fixing the other ones. A single iteration of CP-ALS updates factor matrices by running Equations (3)-(7) once. CP-ALS first finds $\mathbf{Q}_k$ while fixing $\mathbf{H}$, $\mathbf{S}_k$, and $\mathbf{V}$ for all k :

$$\mathbf{A}_k \Sigma \mathbf{B}_k^\top \leftarrow \mathbf{X}_k \mathbf{V} \mathbf{S}_k \mathbf{H}^\top \quad (3)$$

$$\mathbf{Q}_k \leftarrow \mathbf{A}_k \mathbf{B}_k^\top \quad (4)$$

where Equation (3) is the truncated SVD with rank R . Then, CP-ALS updates $\mathbf{H}$, $\{\mathbf{S}_k\}$, and $\mathbf{V}$ as follows:

$$\mathbf{W} \leftarrow \mathbf{Y}_{(3)} (\mathbf{V} \odot \mathbf{H}) (\mathbf{V}^\top \mathbf{V} * \mathbf{H}^\top \mathbf{H})^\dagger \quad (5)$$

$$\mathbf{H} \leftarrow \mathbf{Y}_{(1)} (\mathbf{W} \odot \mathbf{V}) (\mathbf{W}^\top \mathbf{W} * \mathbf{V}^\top \mathbf{V})^\dagger \quad (6)$$

$$\mathbf{V} \leftarrow \mathbf{Y}_{(2)} (\mathbf{W} \odot \mathbf{H}) (\mathbf{W}^\top \mathbf{W} * \mathbf{H}^\top \mathbf{H})^\dagger \quad (7)$$

where the k -th row of $\mathbf{W}_s \in \mathbb{R}^{K \times R}$ is the diagonal entries of $\mathbf{S}_k$, $\mathbf{Y} \in \mathbb{R}^{R \times J \times K}$ is a constructed tensor from slices $\mathbf{Y}_k$ for all k , and $\mathbf{Y}_k \in \mathbb{R}^{R \times J}$ is $\mathbf{Q}_k^\top \mathbf{X}_k$. For each k , $\mathbf{S}_k$ and $\mathbf{U}_k$ are updated as $\text{diag}(\mathbf{W}(k, :))$ and $\mathbf{Q}_k \mathbf{H}$, respectively. CP-ALS repeats Equations (3)-(7) until the results converge.

C. PARAFAC Decomposition

Given a regular tensor $\{\mathbf{X}_k\}_{k=1}^K \in \mathbb{R}^{I \times J}$, PARAFAC [10] decomposes it into a sum of R component tensors, each of which is the outer product of columns from $\mathbf{A}$, $\mathbf{B}$, and $\mathbf{C}$:

$$\{\mathbf{X}_k\}_{k=1}^K \approx \sum_{r=1}^R \mathbf{A}(:, r) \circ \mathbf{B}(:, r) \circ \mathbf{C}(:, r)$$

where $\circ$ denotes the outer product. The objective function of PARAFAC is as follows:

$$\min_{\mathbf{A}, \mathbf{B}, \mathbf{C}} \sum_{i,j,k} \left(\mathbf{X}_k(i, j) - \sum_{r=1}^R \mathbf{A}(i, r) \mathbf{B}(j, r) \mathbf{C}(k, r) \right)^2$$

where $\mathbf{A} \in \mathbb{R}^{I \times R}$, $\mathbf{B} \in \mathbb{R}^{J \times R}$, and $\mathbf{C} \in \mathbb{R}^{K \times R}$ are the latent factor matrices with rank R .

CP-ALS iteratively updates the latent factors while fixing the others factors. A single iteration of CP-ALS is as follows:

$$\mathbf{A} \leftarrow \mathbf{X}_{(1)} (\mathbf{C} \odot \mathbf{B}) [(\mathbf{C} \odot \mathbf{B})^\top (\mathbf{C} \odot \mathbf{B})]^{-1} \quad (8)$$

$$\mathbf{B} \leftarrow \mathbf{X}_{(2)} (\mathbf{C} \odot \mathbf{A}) [(\mathbf{C} \odot \mathbf{A})^\top (\mathbf{C} \odot \mathbf{A})]^{-1} \quad (9)$$

$$\mathbf{C} \leftarrow \mathbf{X}_{(3)} (\mathbf{B} \odot \mathbf{A}) [(\mathbf{B} \odot \mathbf{A})^\top (\mathbf{B} \odot \mathbf{A})]^{-1} \quad (10)$$

where $\mathbf{X}_{(n)}$ is a mode- n matricization of $\{\mathbf{X}_k\}_{k=1}^K$. We repeat Equations (8)-(10) until the results converge.

D. Model-Agnostic Meta-Learning (MAML)

Meta-learning trains a general model with multiple given tasks so that it adapts to a new problem quickly. Meta-learning effectively addresses the problem of limited data. Previous meta-learning methods train the general model using gradient information from the given tasks [15], [16]. Among them, MAML [17] has been widely used due to its robustness.

MAML trains a model on multiple given tasks $\mathcal{T}_d$ for $d = 1, \dots, D$, with the goal of finding a set of initial parameters θ_{meta} that can be fine-tuned to perform well on a new target task quickly and accurately. Note that the objective of the meta model $f_{\theta_{\text{meta}}}$ parameterized by θ_{meta} is not to perform well on the given tasks, but to quickly adapt to a new target task.

For every given task $\mathcal{T}_d$, MAML adapts the meta model parameters θ_{meta} to each $\mathcal{T}_d$ by performing a single (or a few) gradient descent update with the objective function $\mathcal{L}_{\mathcal{T}_d}$:

$$\theta'_d \leftarrow \theta_{\text{meta}} - \alpha \nabla_{\theta_{\text{meta}}} \mathcal{L}_{\mathcal{T}_d}(f_{\theta_{\text{meta}}}) \quad (11)$$

where θ'_d denotes the adapted θ_{meta} to each $\mathcal{T}_d$. This simulates the adaptation process of θ_{meta} to a new target task with a few gradient-descent updates.

MAML then updates θ_{meta} following $\sum_{d=1}^D \mathcal{L}_{\mathcal{T}_d}(f_{\theta'_d})$:

$$\theta_{\text{meta}} \leftarrow \theta_{\text{meta}} - \beta \nabla_{\theta_{\text{meta}}} \sum_{d=1}^D \mathcal{L}_{\mathcal{T}_d}(f_{\theta'_d}). \quad (12)$$

The meta-gradient is a gradient in terms of θ_{meta} whereas the loss is parameterized by θ'_d . This enables θ_{meta} to contain easily transplantable information from all tasks $\mathcal{T}_d$. We denote $\nabla_{\theta_{\text{meta}}} \sum_{d=1}^D \mathcal{L}_{\mathcal{T}_d}(f_{\theta'_d})$, which is a specially designed gradient for meta-learning, as *meta-gradient* in the rest of this paper. We repeat the process until the maximum iteration is reached.

FOMAML (First-Order MAML) [18] is an efficient variant of MAML that approximates the second-order derivatives. The overall procedure of FOMAML is similar to MAML. FOMAML adapts θ_{meta} to each $\mathcal{T}_d$ by performing a single (or a few) gradient-descent update as shown in Equation (11). The difference is that FOMAML approximates the update process of θ_{meta} in Equation (12) as follows:

$$\theta_{\text{meta}} \leftarrow \theta_{\text{meta}} + \alpha \sum_{d=1}^D (\theta_d^{(l)} - \theta_{\text{meta}})$$

where $\theta_d^{(l)}$ is θ_{meta} after l gradient-descent updates on $\mathcal{T}_d$.

E. Problem Definition

Problem 1 (Domain adaptation for irregular tensor decomposition). *We are given a set $\mathcal{S}$ of source domains. For each domain $s \in \mathcal{S}$, there is an irregular tensor $\{\mathbf{X}_s\}$. Then, we have a new irregular tensor $\{\mathbf{X}_T\}$ from an unseen target domain T . The problem aims to quickly find the latent factor matrices of $\{\mathbf{X}_T\}$ that reconstruct $\{\mathbf{X}_T\}$ accurately exploiting the tensors of source domains.*

The objective function $\mathcal{L}$ of Problem 1 is as follows:

$$\min_{\theta_T} \mathcal{L}(\{\mathbf{X}_T\}; \theta_T) = \sum_{k=1}^K \|\mathbf{X}_{T;k} - \mathbf{U}_{T;k} \mathbf{S}_{T;k} \mathbf{V}_T^\top\|_F^2 \quad (13)$$

where $\mathbf{X}_{T;k} \in \mathbb{R}^{I_k \times J}$ is a k -th slice of $\{\mathbf{X}_T\} \in \mathbb{R}^{I \times J \times K}$, and $\theta_T = \{\{\mathbf{U}_T\}, \{\mathbf{S}_T\}, \mathbf{V}_T\}$. Note that the knowledge of source domains is utilized for optimizing the objective $\mathcal{L}$.

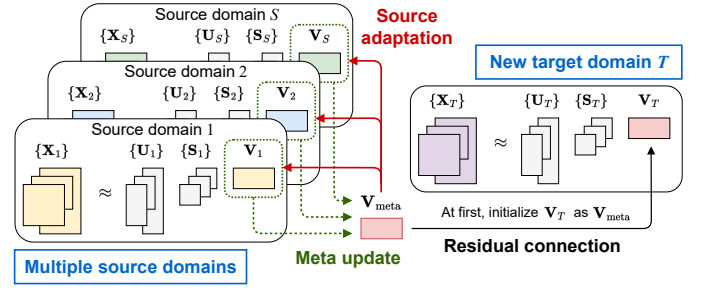


Fig. 2. Overall process of META-P2. We train the easily adaptable meta factor $\mathbf{V}_{\text{meta}}$ from multiple source tensors $\{\mathbf{X}_s\}$ for $s \in \{1, \dots, S\}$. This is done by iteratively performing the source adaptation and meta-update steps. We utilize the trained $\mathbf{V}_{\text{meta}}$ for decomposing a new target tensor $\{\mathbf{X}_T\}$.

We extend Problem 1 to regular tensors by assuming that the tensors $\{\mathbf{X}_s\}$ from all source domains $s \in \mathcal{S}$ and $\{\mathbf{X}_T\}$ from the target domain T are regular tensors:

Problem 2 (Domain adaptation for regular tensor decomposition). *We are given a set $\mathcal{S}$ of source domains where each domain $s \in \mathcal{S}$ is represented by a regular tensor $\{\mathbf{X}_s\}$. From a new target domain T , a regular tensor $\{\mathbf{X}_T\}$ is provided. The objective is to efficiently derive latent factors that reconstruct $\{\mathbf{X}_T\}$ accurately by leveraging the source tensors.*

The objective function $\mathcal{L}_{\text{reg}}(\{\mathbf{X}_{T;k}; \phi_T\})$ of Problem 2 is:

$$\min_{\phi_T} \sum_{i,j,k} \left(\mathbf{X}_{T;k}(i,j) - \sum_{r=1}^R \mathbf{A}_T(i,r) \mathbf{B}_T(j,r) \mathbf{C}_T(k,r) \right)^2 \quad (14)$$

where $\mathbf{X}_{T;k} \in \mathbb{R}^{I \times J}$ is the k -th slice of $\{\mathbf{X}_T\} \in \mathbb{R}^{I \times J \times K}$ and $\phi_T = \{\mathbf{A}_T, \mathbf{B}_T, \mathbf{C}_T\}$ is a set of factors.

Problems 1 and 2 are addressed in Sections III and IV, respectively.

III. PROPOSED METHOD FOR IRREGULAR TENSORS

We propose META-P2, a fast and accurate domain adaptation method for irregular tensors. The main challenges and our approaches are as follows:

- 1) **How can we utilize given source irregular tensors for decomposing a new irregular tensor in a target domain quickly and accurately?** We learn a general *meta factor matrix* from given source domains, which is easily transferred to the new target domain (Section III-A).
- 2) **How can we learn the meta factor matrix from source domains which is easily transferred to the target domain?** We perform *meta-learning-based PARAFAC2 decomposition* on source domains. This is achievable by carefully modeling the gradients of reconstruction error in terms of the meta factor matrix (Section III-B).
- 3) **How can we utilize the trained meta factor matrix when decomposing a new tensor of a target domain?** We adapt the meta factor to the target domain while preventing the decomposed results of the target domain from completely forgetting the information of source domains (Section III-C).

A. Meta Factor Matrix of META-P2

In domain adaptation for irregular tensor decomposition, the objective is to quickly find accurate latent factor matrices of

Algorithm 1: Computing meta factor matrix $\mathbf{V}_{\text{meta}}$ of META-P2 from multiple source domains.

input : Irregular tensors $\{\mathbf{X}_{s;k}\}_{k=1}^K \in \mathbb{R}^{I_k \times J \times K}$ from all source domains $s = 1, \dots, S$
output : Meta factor matrix $\mathbf{V}_{\text{meta}} \in \mathbb{R}^{J \times R}$
parameters: Target rank R

```

1 Randomly initialize  $\mathbf{W}_s \in \mathbb{R}^{K \times R}$ ,  $\mathbf{H}_s \in \mathbb{R}^{R \times R}$  for all  $s = 1, \dots, S$ , and  $\mathbf{V}_{\text{meta}}$ ;
2 repeat
  /* Source adaptation step */
3   for  $s=1, \dots, S$  do
4     Adapt  $\mathbf{V}_{\text{meta}}$  to domain  $s$  by an ALS iteration, and
      save the result as  $\mathbf{V}_s$ ; // Equations (19), (20)
5   end
  /* Meta-update step */
6   Update  $\mathbf{V}_{\text{meta}}$  along the meta-gradient, which is
      computed by aggregating all  $\mathbf{V}_s$ ; // Equation (23)
7 until the maximum iteration is reached;
```

a new irregular tensor $\{\mathbf{X}_T\}$ of the target domain T , using the given irregular tensors $\{\mathbf{X}_s\}$ in multiple source domains $s \in \mathcal{S}$. Specifically, we aim to accurately decompose the target $\{\mathbf{X}_T\}$ with limited information in a few ALS iteration updates. To do this, it is important to define the information from source domains that helps decompose the target tensor. If the information is slice-specific or domain-specific, it cannot directly be used for analyzing the new target tensor.

For each tensor $\{\mathbf{X}_s\}$ in source domain s , the decomposed results are represented by the multiplication of three components: $\{\mathbf{U}_{s;k}\}$, $\{\mathbf{S}_{s;k}\}$, and $\mathbf{V}_s$. Each k -th slice $\mathbf{X}_{s;k}$ of $\{\mathbf{X}_s\}$ is decomposed into $\mathbf{U}_{s;k}\mathbf{S}_{s;k}\mathbf{V}_s$. Note that $\mathbf{V}_s$ contains representative information of domain s since $\mathbf{V}_s$ is used for constructing every slice of $\{\mathbf{X}_s\}$, while $\{\mathbf{U}_{s;k}\}$ and $\{\mathbf{S}_{s;k}\}$ are specific to each slice.

We propose to share $\mathbf{V}_s$ along the source domains, which we denote as $\mathbf{V}_{\text{meta}}$, and utilize it for analyzing the target domain. The intuition is to learn *general* information from the source domains by forcing $\mathbf{V}_{\text{meta}}$ to reconstruct every source tensor well. A challenge is that naively sharing $\mathbf{V}_{\text{meta}}$ only aims to reconstruct the source tensors, and fails to achieve accurate results on the target domain when the target has unseen properties in source domains.

To address this, META-P2 enforces $\mathbf{V}_{\text{meta}}$ to possess an additional property: *easily adaptable*. We learn $\mathbf{V}_{\text{meta}}$ to quickly and accurately adapt to any new domain, not only to the given source domains. This is done by training $\mathbf{V}_{\text{meta}}$ based on the meta-learning scheme. As a result, META-P2 learns *general* and *easily adaptable* meta factor matrix $\mathbf{V}_{\text{meta}}$ from the source tensors.

The overall process of META-P2 is illustrated in Fig. 2. We first compute the meta factor $\mathbf{V}_{\text{meta}}$ from multiple source domains. Then we leverage the trained $\mathbf{V}_{\text{meta}}$ for decomposing a new target tensor $\{\mathbf{X}_T\}$. This enables us to effectively utilize the useful information of source domains for analyzing the target tensor without decomposing all the domains together.

B. Decomposition of META-P2 on Source Domains

META-P2 learns the *general* and *easily adaptable* meta factor $\mathbf{V}_{\text{meta}}$ from the multiple source tensors based on the

meta-learning scheme. The meta-learning aims to train a general model that adapts to a new task quickly and accurately utilizing multiple source tasks. The meta-learning based decomposition of META-P2 in the source domains consists of two steps: a) source adaptation and b) meta-update steps. The source adaptation step adapts $\mathbf{V}_{\text{meta}}$ to every source domain. Using the adapted results, the meta-update step finds $\mathbf{V}_{\text{meta}}$ that is prepared for adaptation to any new domain, rather than accurate reconstruction of the source tensors. We repeat those two steps until the maximum iteration is reached. Algorithm 1 shows the overall process of decomposition in source domains.

Using the objective function $\mathcal{L}$ of META-P2 in Equation (13), the source adaptation of $\mathbf{V}_{\text{meta}}$ for each source domain s formulated as follows:

$$\mathbf{V}_s = \mathbf{V}_{\text{meta}} - \eta \nabla_{\mathbf{V}_{\text{meta}}} \sum_{k=1}^K \|\mathbf{X}_{s;k} - \mathbf{U}_{s;k}\mathbf{S}_{s;k}\mathbf{V}_{\text{meta}}^\top\|_F^2 \quad (15)$$

where η is a learning rate. To remove the constant term $\mathbf{U}_{s;k}\mathbf{S}_{s;k}$ in front of $\mathbf{V}_{\text{meta}}$, we rewrite $\mathcal{L}(\{\mathbf{X}_s\}; \mathbf{V}_{\text{meta}})$ as

$$\mathcal{L}_V(\{\mathbf{X}_s\}; \mathbf{V}_{\text{meta}}) = \sum_{k=1}^K \|\mathbf{E}_{s;k}\mathbf{X}_{s;k} - \mathbf{V}_{\text{meta}}^\top\|_F^2 \quad (16)$$

where $\mathbf{E}_{s;k} = K(\sum_{j=1}^K \mathbf{S}_{s;j}^\top \mathbf{U}_{s;j}^\top \mathbf{U}_{s;j} \mathbf{S}_{s;j})^{-1} \mathbf{S}_{s;k}^\top \mathbf{U}_{s;k}^\top$. Note that minimizing $\mathcal{L}_V$ in Equation (16) is equivalent to minimizing $\mathcal{L}$ in Equation (13) for training $\mathbf{V}_{\text{meta}}$ (Lemma 6 in Section B of Supplementary Material). Then the source adaptation step of META-P2 is expressed as

$$\mathbf{V}_s = \mathbf{V}_{\text{meta}} - \eta \nabla_{\mathbf{V}_{\text{meta}}} \mathcal{L}_V(\{\mathbf{X}_s\}; \mathbf{V}_{\text{meta}}). \quad (17)$$

The meta-update step of $\mathbf{V}_{\text{meta}}$ is as follows:

$$\mathbf{V}_{\text{meta}} \leftarrow \mathbf{V}_{\text{meta}} - \eta \nabla_{\mathbf{V}_{\text{meta}}} \sum_{s \in \mathcal{S}} \mathcal{L}_V(\{\mathbf{X}_s\}; \mathbf{V}_s). \quad (18)$$

However, CP-ALS algorithm is widely used for learning the latent factors of irregular tensors [19]–[21] compared to the gradient-descent one. This is mainly because the ALS-based decomposition gives accurate factors in a few iterations and guarantees the uniqueness of the results [5]. In the following Sections III-B1 and III-B2, we describe how we design the gradient-based meta-learning by the ALS-based meta-learning.

1) Source adaptation step: In the source adaptation step, we adapt $\mathbf{V}_{\text{meta}}$ to each source domain by performing a single ALS iteration instead of the single gradient-based update in Equation (17). The idea stems from the observation that both gradient-based and ALS-based training finds the optimal parameter by gradually minimizing the objective function. This idea also aligns with the typical meta-learning scenario which constrains the numbers of gradient-descent updates to adapt the meta model parameters to each given task. To perform the ALS-based adaptation of $\mathbf{V}_{\text{meta}}$, we first update $\mathbf{W}_s$ and $\mathbf{H}_s$ using Equations (5) and (6), respectively:

$$\mathbf{W}_s = (\mathbf{Y}_s)_{(3)} (\mathbf{V}_{\text{meta}} \odot \mathbf{H}_s) (\mathbf{V}_{\text{meta}}^\top \mathbf{V}_{\text{meta}} * \mathbf{H}_s^\top \mathbf{H}_s)^\dagger \quad (19)$$

$$\mathbf{H}_s = (\mathbf{Y}_s)_{(1)} (\mathbf{W}_s \odot \mathbf{V}_{\text{meta}}) (\mathbf{W}_s^\top \mathbf{W}_s * \mathbf{V}_{\text{meta}}^\top \mathbf{V}_{\text{meta}})^\dagger$$

where $\mathbf{Y}_s \in \mathbb{R}^{R \times J \times K}$ is constructed from slices $\mathbf{Y}_{s;k} = \mathbf{Q}_{s;k}^\top \mathbf{X}_{s;k}$ for all $k = 1, \dots, K$, and $\mathbf{Q}_{s;k} \in \mathbb{R}^{I_k \times R}$ is computed using Equation (4). The k -th row of $\mathbf{W}_s \in \mathbb{R}^{K \times R}$ corresponds to the diagonal entries of $\mathbf{S}_k$. Then we adapt $\mathbf{V}_{\text{meta}}$ to each domain s using Equation (7):

$$\mathbf{V}_s = (\mathbf{Y}_s)_{(2)} (\mathbf{W}_s \odot \mathbf{H}_s) (\mathbf{W}_s^\top \mathbf{W}_s * \mathbf{H}_s^\top \mathbf{H}_s)^\dagger. \quad (20)$$

Note that we express the gradient-based source adaptation step in Equation (17) by an ALS-based adaptation in Equation (20).

2) **Meta-update step:** In the meta-update step, we update the meta factor matrix $\mathbf{V}_{\text{meta}}$ to perform well after adapting to any new target domain, rather than generally performing well across all source domains. Our approach for designing the gradient-based meta-update in Equation (18) by an ALS-based meta-update for META-P2 is to express the *meta-gradient* $\nabla_{\mathbf{V}_{\text{meta}}} \sum_{s \in \mathcal{S}} \mathcal{L}_V(\{\mathbf{X}_s\}; \mathbf{V}_s)$ using the factors computed by ALS iterations. However, directly differentiating $\mathcal{L}_V(\{\mathbf{X}_s\}; \mathbf{V}_s)$ in terms of $\mathbf{V}_{\text{meta}}$ is challenging since $\mathbf{V}_s$ is not parameterized by $\mathbf{V}_{\text{meta}}$.

To address this, we break down $\nabla_{\mathbf{V}_{\text{meta}}} \mathcal{L}_V(\{\mathbf{X}_s\}; \mathbf{V}_s)$ of the meta-gradient into $\nabla_{\mathbf{V}_{\text{meta}}} \mathcal{L}_V(\{\mathbf{X}_s\}; \mathbf{V}_{\text{meta}})$ and $\nabla_{\mathbf{V}_s} \mathcal{L}_V(\{\mathbf{X}_s\}; \mathbf{V}_s)$. In the following, we first describe how we express the two gradients using only the factors computed by ALS iterations, and then we show the relationship between the meta-gradient and the two gradients.

a) **Two Gradients:** Let $\mathbf{V}'_s$ denote $\mathbf{V}_s$ adapted to domain s again:

$$\mathbf{V}'_s = \mathbf{V}_s - \eta \nabla_{\mathbf{V}_s} \mathcal{L}_V(\{\mathbf{X}_s\}; \mathbf{V}_s) \quad (21)$$

$$= (\mathbf{Y}_s)_{(2)} (\mathbf{W}'_s \odot \mathbf{H}'_s) (\mathbf{W}'_s{}^\top \mathbf{W}'_s * \mathbf{H}'_s{}^\top \mathbf{H}'_s)^\dagger \quad (22)$$

where $\mathbf{W}'_s$ and $\mathbf{H}'_s$ denote $\mathbf{W}_s$ and $\mathbf{H}_s$ after performing an additional ALS updates in Equation (19) while replacing $\mathbf{V}_{\text{meta}}$ with $\mathbf{V}_s$. The second equality uses our approximation in the source adaptation step that a single gradient-descent update is equal to a single ALS iteration update (see Equations (17) and (20)). Then the two gradients $\nabla_{\mathbf{V}_{\text{meta}}} \mathcal{L}_V(\{\mathbf{X}_s\}; \mathbf{V}_{\text{meta}})$ and $\nabla_{\mathbf{V}_s} \mathcal{L}_V(\{\mathbf{X}_s\}; \mathbf{V}_s)$ are simply expressed by $(\mathbf{V}_{\text{meta}} - \mathbf{V}_s)/\eta$ and $(\mathbf{V}_s - \mathbf{V}'_s)/\eta$ using Equations (17) and (21), respectively. Note that $\mathbf{V}_s$ and $\mathbf{V}'_s$ are computed by the ALS iterations in Equations (20) and (22).

b) **Relationship between Meta-gradient and the Two Gradients:** Lemma 1 shows the relationship between each term $\nabla_{\mathbf{V}_{\text{meta}}} \mathcal{L}_V(\{\mathbf{X}_s\}; \mathbf{V}_s)$ of the meta-gradient and the two gradients $\nabla_{\mathbf{V}_{\text{meta}}} \mathcal{L}_V(\{\mathbf{X}_s\}; \mathbf{V}_{\text{meta}})$ and $\nabla_{\mathbf{V}_s} \mathcal{L}_V(\{\mathbf{X}_s\}; \mathbf{V}_s)$.

Lemma 1. *Each term $\nabla_{\mathbf{V}_{\text{meta}}} \mathcal{L}_V(\{\mathbf{X}_s\}; \mathbf{V}_s)$ of the meta-gradient is proportional to the sum of the two gradients $\nabla_{\mathbf{V}_{\text{meta}}} \mathcal{L}_V(\{\mathbf{X}_s\}; \mathbf{V}_{\text{meta}})$ and $\nabla_{\mathbf{V}_s} \mathcal{L}_V(\{\mathbf{X}_s\}; \mathbf{V}_s)$:*

$$\nabla_{\mathbf{V}_{\text{meta}}} \mathcal{L}_V(\{\mathbf{X}_s\}; \mathbf{V}_s) = C (\nabla_{\mathbf{V}_{\text{meta}}} \mathcal{L}_V(\{\mathbf{X}_s\}; \mathbf{V}_{\text{meta}}) + \nabla_{\mathbf{V}_s} \mathcal{L}_V(\{\mathbf{X}_s\}; \mathbf{V}_s))$$

where $C = (1 - 2\eta K)^2 / (2(1 - \eta K))$, η is the learning rate, and K is the number of slices in each source domain.

Proof. Section C of Supplementary Material for proof. $\square$

Using Lemma 1, we derive Theorem 1 which expresses the meta-update step of META-P2 in terms of $\mathbf{V}'_s$ that is computed by the ALS iteration in Equation (22).

Theorem 1. *The meta-update step of META-P2 in Equation (18) is expressed in terms of $\mathbf{V}'_s$ that is computed using the ALS iterations:*

$$\mathbf{V}_{\text{meta}} \leftarrow \mathbf{V}_{\text{meta}} + C \sum_{s \in \mathcal{S}} (\mathbf{V}'_s - \mathbf{V}_{\text{meta}}).$$

$C = (1 - 2\eta K)^2 / (2(1 - \eta K))$ is a constant with learning rate η and the number K of slices in each domain.

Proof. Section D of Supplementary Material for proof. $\square$

We choose η that satisfies $C = 1/|\mathcal{S}|$. As a result, the meta-update step of $\mathbf{V}_{\text{meta}}$ is simply expressed by the mean of $\mathbf{V}'_s$

Algorithm 2: Tensor decomposition of META-P2 on a new target tensor exploiting the meta factor matrix.

input : Irregular tensor $\mathbf{X}_T \in \mathbb{R}^{I_k \times J \times K}$, and a trained meta factor $\mathbf{V}_{\text{meta}} \in \mathbb{R}^{J \times R}$

output : Decomposed matrices $\mathbf{U}_{T;k} \in \mathbb{R}^{I_k \times R}$ and $\mathbf{S}_{T;k} \in \mathbb{R}^{R \times R}$ for all k , and $\mathbf{V}_T$

parameters: Target rank R

- 1 Randomly initialize $\mathbf{H}_T \in \mathbb{R}^{R \times R}$, and $\mathbf{S}_{T;k}$ for all k ;
- 2 Initialize $\mathbf{V}_T$ with $\mathbf{V}_{\text{meta}}$;
- 3 **repeat**
- 4 | Update $\{\mathbf{Q}_T\}, \mathbf{H}_T, \mathbf{W}_T, \mathbf{V}_T$ using Equations (3)-(7);
- 5 | $\mathbf{V}_T \leftarrow (\mathbf{V}_T + \mathbf{V}_{\text{meta}})/2$; // residual conn.
- 6 **until** the maximum iteration is reached;
- 7 **for** $k=1, \dots, K$ **do**
- 8 | $\mathbf{S}_{T;k} \leftarrow \text{diag}(\mathbf{W}_T(k, :));$
- 9 | $\mathbf{U}_{T;k} \leftarrow \mathbf{Q}_{T;k} \mathbf{H}_T$;
- 10 **end**

for all source domains $s \in \mathcal{S}$ as follows:

$$\mathbf{V}_{\text{meta}} \leftarrow \mathbf{V}_{\text{meta}} + \frac{1}{|\mathcal{S}|} \sum_{s \in \mathcal{S}} (\mathbf{V}'_s - \mathbf{V}_{\text{meta}}) = \frac{1}{|\mathcal{S}|} \sum_{s \in \mathcal{S}} \mathbf{V}'_s. \quad (23)$$

Note that we express the meta-update step of META-P2 in Equation (18) using only $\mathbf{V}'_s$, which is computed by performing ALS iterations. We learn $\mathbf{V}_{\text{meta}}$ from the source tensors by repeatedly performing the source adaptation step in Equation (20) and the meta-update step in Equation (23).

C. Inference of META-P2 on Target Domain

META-P2 analyzes a new irregular tensor $\{\mathbf{X}_T\}$ of target domain T using the patterns of source domains. The patterns are represented as $\mathbf{V}_{\text{meta}}$. The main idea is to initialize the latent factor $\mathbf{V}_T$ with $\mathbf{V}_{\text{meta}}$, and find the adapted factor matrices to the target domain by running a few ALS iterations. The decomposed results are utilized for performing many downstream tasks on the target domain including missing value prediction, anomaly detection, feature extraction, etc.

For performing a downstream task, overfitting the latent factors to the target domain degrades the performance when there are only a few entries in the target tensor. To prevent forgetting the trained patterns $\mathbf{V}_{\text{meta}}$ of source domains, we exploit a residual connection between $\mathbf{V}_{\text{meta}}$ and $\mathbf{V}_T$.

When decomposing a new target tensor, abnormal values such as anomalies and missing entries deteriorate the quality of the decomposed results. Since a single ALS step of META-P2 reconstructs the target tensor accurately using the general information $\mathbf{V}_{\text{meta}}$ of source domains, META-P2 imputes the abnormal values with the corresponding reconstructed values after a single ALS step. For example, when there are anomalies in the target tensor, META-P2 replaces the detected anomalous entries with the reconstructed values after a single ALS step.

D. Theoretical Analysis on META-P2

We theoretically show that META-P2 has the same time complexity to that of a naive PARAFAC2.

Lemma 2. *The decomposition of META-P2 on source domains (Algorithm 1) has the same time complexity as that of a naive PARAFAC2 decomposition on the source domains.*

Algorithm 3: Workflow of META-P on regular tensors.

```

input      : Regular tensors  $\{\mathbf{X}_{s;k}\}_{k=1}^K \in \mathbb{R}^{I \times J \times K}$  from
               all source domains  $s = 1, \dots, S$ , regular tensor
                $\{\mathbf{X}_{T;k}\}_{k=1}^K \in \mathbb{R}^{I \times J \times K}$  from target domain  $T$ 
output    : Factor matrices  $\mathbf{A}_T \in \mathbb{R}^{I \times R}$ ,  $\mathbf{B}_T \in \mathbb{R}^{J \times R}$ ,
               and  $\mathbf{C}_T \in \mathbb{R}^{K \times R}$  of target domain  $T$ 
parameters: Target rank  $R$ 

/* Decomposition on source domains */
1 Randomly initialize  $\mathbf{B}_{\text{meta}}$  and  $\mathbf{C}_{\text{meta}}$ ;
2 repeat
   /* Source adaptation step */
   3 for  $s=1, \dots, S$  do
     4   Adapt  $\mathbf{A}_{\text{meta}}$ ,  $\mathbf{B}_{\text{meta}}$ , and  $\mathbf{C}_{\text{meta}}$  to domain  $s$ , and
         save the result as  $\mathbf{A}_s$ ,  $\mathbf{B}_s$ , and  $\mathbf{C}_s$ , respectively;
         // Equation (24)
   5 end
   /* Meta-update step */
   6 Update  $\mathbf{A}_{\text{meta}}$ ,  $\mathbf{B}_{\text{meta}}$ , and  $\mathbf{C}_{\text{meta}}$  by aggregating all
        $\mathbf{A}_s$ ,  $\mathbf{B}_s$ , and  $\mathbf{C}_s$ , respectively; // Equation (25)
7 until the maximum iteration is reached;

/* Decomposition on a new target domain */
8  $\mathbf{A}_T, \mathbf{B}_T, \mathbf{C}_T \leftarrow \mathbf{A}_{\text{meta}}, \mathbf{B}_{\text{meta}}, \mathbf{C}_{\text{meta}}$ ;
9 repeat
10  Update  $\mathbf{A}_T$ ,  $\mathbf{B}_T$ , and  $\mathbf{C}_T$ ; // Equations (8)-(10)
11   $\mathbf{A}_T \leftarrow (\mathbf{A}_T + \mathbf{A}_{\text{meta}})/2$ ;
12   $\mathbf{B}_T \leftarrow (\mathbf{B}_T + \mathbf{B}_{\text{meta}})/2$ ;
13   $\mathbf{C}_T \leftarrow (\mathbf{C}_T + \mathbf{C}_{\text{meta}})/2$ ;
14 until the maximum iteration is reached;

```

Proof. Section E of Supplementary Material for proof. $\square$

Lemma 3. *Given the meta factor matrix $\mathbf{V}_{\text{meta}}$, the time complexity of META-P2 for finding latent factors of a target tensor (Algorithm 2) is the same as that of performing PARAFAC2 decomposition on the target tensor.*

Proof. Section F of Supplementary Material for proof. $\square$

IV. PROPOSED METHOD FOR REGULAR TENSORS

Regular tensors are a special case of irregular tensors, where all dimensions have the same length. We extend META-P2 to regular tensors, referring to the extension as META-P. Crucially, META-P is not a simple port: it introduces a new theoretical result (Section IV-C), and yields consistent improvements in accuracy (Tables III, IV) and adaptation speed (Fig. 4).

The main challenges and our approaches are as follows:

- 1) **How can we redefine the meta factor of META-P2 for regular tensors in META-P?** Since there is no slice-specific latent factors in regular tensor decomposition, META-P shares all latent factors across source domains and defines them as meta factors (Section IV-A).
- 2) **How can we grant easy adaptability to the meta factors of META-P?** We perform meta-learning-based PARAFAC on source domains (Section IV-B).

A. Meta Factors of META-P

We generalize META-P2 to regular tensors. Specifically, we extend META-P2 to PARAFAC, which is a generalized form of PARAFAC2 to regular tensors. META-P follows the overall process of META-P2. First, META-P learns *general* and *easily*

adaptable meta factors from the source domains. For a new regular tensor from a target domain, META-P leverages the trained meta factors to analyze it quickly and accurately. The overall process is described in Algorithm 3.

A main challenge is that the latent factors in PARAFAC2 and PARAFAC are fundamentally different. PARAFAC2 decomposes an irregular tensor $\{\mathbf{X}_{s;k}\} \in \mathbb{R}^{I_k \times J \times K}$ of domain s into slice-specific factors $\{\mathbf{U}_{s;k}\}$ and $\{\mathbf{S}_{s;k}\}$, and a domain-specific factor $\mathbf{V}_{s;k}$. In contrast, PARAFAC decomposes a regular tensor $\{\mathbf{X}_d\} \in \mathbb{R}^{I \times J \times K}$ into three domain-specific factor matrices, $\mathbf{A}_s$, $\mathbf{B}_d$, and $\mathbf{C}_d$, without slice-specific factors.

Thus, we redefine the meta factor matrix for regular tensors. We share all the latent factors in PARAFAC, which naturally imparts the first key property of meta factors: *generality*. We denote the shared meta factors as $\mathbf{A}_{\text{meta}}$, $\mathbf{B}_{\text{meta}}$, and $\mathbf{C}_{\text{meta}}$.

B. Decomposition of META-P on Source Domains

META-P performs meta-learning-based PARAFAC decomposition on source domains to pre-learn general and easily adaptable meta factors. Similar to META-P2, the decomposition involves two steps: a) source adaptation and b) meta-update. In the first step, meta factors are adapted to each source domain. The meta-update step then carefully aggregates these adapted factors to refine the meta factors.

1) **Source adaptation of META-P:** The meta factors $\mathbf{A}_{\text{meta}}$, $\mathbf{B}_{\text{meta}}$, and $\mathbf{C}_{\text{meta}}$ are adapted to each source domain s by performing a single ALS iteration, similar to Equations (19)-(20) in META-P2:

$$\begin{aligned}
 \mathbf{A}_s &= (\mathbf{X}_s)_{(1)} \mathbf{Z}_1 (\mathbf{Z}_1^\top \mathbf{Z}_1)^{-1} \quad \text{where } \mathbf{Z}_1 = \mathbf{C}_{\text{meta}} \odot \mathbf{B}_{\text{meta}} \\
 \mathbf{B}_s &= (\mathbf{X}_s)_{(2)} \mathbf{Z}_2 (\mathbf{Z}_2^\top \mathbf{Z}_2)^{-1} \quad \text{where } \mathbf{Z}_2 = \mathbf{C}_{\text{meta}} \odot \mathbf{A}_s \\
 \mathbf{C}_s &= (\mathbf{X}_s)_{(3)} \mathbf{Z}_3 (\mathbf{Z}_3^\top \mathbf{Z}_3)^{-1} \quad \text{where } \mathbf{Z}_3 = \mathbf{B}_s \odot \mathbf{A}_s.
 \end{aligned} \tag{24}$$

Notice that sharing the meta factors as initial points across the source domains imparts *generality* to the meta factors.

2) **Meta-update of META-P:** We update the meta-factors to ensure high performance after adaptation to any new target domain, rather than focusing on general performance across all source domains. The meta-factors are updated in a manner similar to Equation (23) of META-P2:

$$\begin{aligned}
 \mathbf{A}_{\text{meta}} &\leftarrow 1/|\mathcal{S}| \sum_{s \in \mathcal{S}} \mathbf{A}'_s \\
 \mathbf{B}_{\text{meta}} &\leftarrow 1/|\mathcal{S}| \sum_{s \in \mathcal{S}} \mathbf{B}'_s \\
 \mathbf{C}_{\text{meta}} &\leftarrow 1/|\mathcal{S}| \sum_{s \in \mathcal{S}} \mathbf{C}'_s
 \end{aligned} \tag{25}$$

where $\mathbf{A}'_s$, $\mathbf{B}'_s$, and $\mathbf{C}'_s$ denote $\mathbf{A}_s$, $\mathbf{B}_s$, and $\mathbf{C}_s$ adapted to domain s again, respectively:

$$\begin{aligned}
 \mathbf{A}'_s &= (\mathbf{X}_s)_{(1)} \mathbf{Z}_1 (\mathbf{Z}_1^\top \mathbf{Z}_1)^{-1} \quad \text{where } \mathbf{Z}_1 = \mathbf{C}_s \odot \mathbf{B}_s \\
 \mathbf{B}'_s &= (\mathbf{X}_s)_{(2)} \mathbf{Z}_2 (\mathbf{Z}_2^\top \mathbf{Z}_2)^{-1} \quad \text{where } \mathbf{Z}_2 = \mathbf{C}_s \odot \mathbf{A}'_s \\
 \mathbf{C}'_s &= (\mathbf{X}_s)_{(3)} \mathbf{Z}_3 (\mathbf{Z}_3^\top \mathbf{Z}_3)^{-1} \quad \text{where } \mathbf{Z}_3 = \mathbf{B}'_s \odot \mathbf{A}'_s.
 \end{aligned} \tag{26}$$

In the following Section IV-C, we theoretically analyze the connection between META-P's decomposition on source domains and FOMAML [18], which is a simplified variant of MAML that reduces computational complexity by avoiding second-order derivatives (refer to Section II-D).

C. Theoretical Analysis on META-P

1) **Relationship between META-P and meta learning:** Unlike META-P2, META-P applies source adaptation and meta-update to all latent factors. Noting that the meta-learning

also applies the processes to all parameters, we discuss the relationship between META-P and FOMAML (First-Order MAML) [18]. We apply the same assumption of META-P2 to META-P that a single gradient-descent update is equal to a single ALS iteration update (see Equations (17) and (20)):

$$\begin{aligned} \mathbf{A}_s &= (\mathbf{X}_s)_{(1)} \mathbf{Z}_1 (\mathbf{Z}_1^\top \mathbf{Z}_1)^{-1} \approx \mathbf{A}_{\text{meta}} - \eta \nabla_{\mathbf{A}_{\text{meta}}} \mathcal{L}_{\text{reg}}(\{\mathbf{X}_s\}; \mathbf{A}_{\text{meta}}) \\ \mathbf{B}_s &= (\mathbf{X}_s)_{(2)} \mathbf{Z}_2 (\mathbf{Z}_2^\top \mathbf{Z}_2)^{-1} \approx \mathbf{B}_{\text{meta}} - \eta \nabla_{\mathbf{B}_{\text{meta}}} \mathcal{L}_{\text{reg}}(\{\mathbf{X}_s\}; \mathbf{B}_{\text{meta}}) \\ \mathbf{C}_s &= (\mathbf{X}_s)_{(3)} \mathbf{Z}_3 (\mathbf{Z}_3^\top \mathbf{Z}_3)^{-1} \approx \mathbf{C}_{\text{meta}} - \eta \nabla_{\mathbf{C}_{\text{meta}}} \mathcal{L}_{\text{reg}}(\{\mathbf{X}_s\}; \mathbf{C}_{\text{meta}}^{(27)}) \end{aligned}$$

where $\mathbf{Z}_1 = \mathbf{C}_{\text{meta}} \odot \mathbf{B}_{\text{meta}}$, $\mathbf{Z}_2 = \mathbf{C}_{\text{meta}} \odot \mathbf{A}_s$, $\mathbf{Z}_3 = \mathbf{B}_s \odot \mathbf{A}_s$, and $\mathcal{L}_{\text{reg}}$ is the objective function of META-P in Equation (14). Then we derive Theorem 2 which shows the relationship between META-P and meta-learning.

Theorem 2. *Given the assumption in Equation (27), the decomposition of META-P on source domains is a special case of FOMAML.*

Proof. Section I of Supplementary Material for proof. $\square$

By Theorem 2, applying l inner ALS updates per source is equivalent to using l inner steps in the FOMAML to adapt a model to a source domain. Consequently, l is a tunable hyperparameter governing per-source adaptation.

Although we provide theoretical connections between our methods and meta-learning frameworks such as MAML (Section III-B) and FOMAML (Theorem 2), deriving formal generalization bounds or convergence guarantees for ALS-based meta-adaptation remains an important open problem. We leave this as a promising direction for future work.

2) **Time complexity of META-P:** We theoretically show that META-P has the same time complexity of PARAFAC.

Lemma 4. *The computation per iteration of META-P on source domains (lines 1-7 in Algorithm 3) has the same time complexity as running a single CP-ALS on the source domains.*

Proof. Section G of Supplementary Material for proof. $\square$

Lemma 5. *The time complexity of META-P for finding latent factors of a target tensor (lines 8-14 Algorithm 3) is the same as that of performing PARAFAC on the target tensor.*

Proof. Section H of Supplementary Material for proof. $\square$

V. EXPERIMENTS

We perform experiments to answer the following questions.

- Q1. Missing Value Prediction.** How accurately do META-P2 and META-P predict missing values of a new target tensor using existing source tensors (Section V-B)?
- Q2. Anomaly Detection.** Are META-P2 and META-P superior to the baselines in detecting anomalies of a new target tensor exploiting the source tensors (Section V-C)?
- Q3. Domain Adaptation Speed.** How fast does the proposed method adapt to the new target domain (Section V-D)?
- Q4. Ablation Study.** Does each step contribute to the performance of the downstream task (Section V-E)?
- Q5. Case Study.** What are the discoveries of analyzing real world data using the proposed method (Section V-F)?

A. Experimental Settings

1) **Datasets:** To evaluate META-P2, we perform experiments on five real-world datasets. For META-P, we use four regular tensor datasets collected from various sensors.

Nasdaq, S&P 500, and Korea stock are stock datasets which contain selected features of daily stock prices. Stocks are grouped into sectors, and we treat their sectors as domains. In each sector, the data are in the form of (*day, feature, stock*); a stock corresponds to a slice matrix of size (*day, feature*).

NATOPS [22] and Cricket [23] are human activity datasets recorded with various sensors. We treat participants as domains since each person has different physical conditions and unique habits. For each person, actions are represented as (*timeframe, sensor, action*); an action corresponds to a slice matrix whose form is (*timeframe, sensor*). Cricket captures umpires performing twelve signals used in cricket games. We treat each signal type, such as No Ball or Dead Ball, as a domain. For each type of signal, the signals are in the form of (*timeframe, sensor, person*); a person corresponds to a slice, and its form is (*timeframe, sensor*). Articular [24], Finger [25], and Urbansound [26] are regular tensor datasets recorded with various sensors. In these datasets, we treat the classes (e.g., type of urban sound) as domains. For each class, movements or sounds are represented as (*timeframe, sensor, movement/sound*); each movement or sound corresponds to a slice whose form is (*timeframe, sensor*). PEMS [27] contains daily traffic data from California Department of Transportation. We treat each day of the week as domain. Additional details on datasets such as data statistics are provided in Supplementary Material (Section J).

2) **Baselines:** We compare META-P2 with previous approaches for irregular tensor decomposition. We do not compare META-P2 with COPA [21] and SPARTan [20] that aim to improve the scalability of the original PARAFAC2 since our objective is to find accurate latent factors of the target irregular tensor with limited number of slices. Transfer learning methods such as parameter fine-tuning or representation reuse are not directly applicable in our setting, as our task involves decomposing newly introduced tensors rather than transferring learned weights across models.

PARAFAC2 [5] performs PARAFAC2 decomposition on the whole domains including source and target. The method optimizes the latent factor matrices based on ALS approach. PARAFAC2-T performs PARAFAC2 decomposition on the target domain only. PARAFAC2-I initializes the factor matrix $\mathbf{V}$ of the target domain with the result of a naive PARAFAC2 decomposition on source domains. AO-ADMM [28] is a recent state-of-the-art decomposition method for irregular tensors. AO-ADMM employs an alternating optimization (AO) approach in conjunction with the alternating direction method of multipliers (ADMM) for fitting PARAFAC2.

For META-P, we compare it with existing regular tensor decomposition methods. PARAFAC [10] applies PARAFAC decomposition to both the source and target domains, optimizing latent factors through ALS iterations. PARAFAC-T performs PARAFAC decomposition on the target domain only. PARAFAC-I initializes the latent factors of the target domain

TABLE I
MISSING VALUE PREDICTION ERROR OF META-P2 AND BASELINES. THE LOWEST ERRORS ARE IN BOLD. META-P2 SHOWS THE BEST PERFORMANCE.

Missing Value Prediction										
Models	Missing ratio $r_m = 0.1$					Missing ratio $r_m = 0.3$				
	Nasdaq	S&P 500	Korea stock	NATOPS	Cricket	Nasdaq	S&P 500	Korea stock	NATOPS	Cricket
PARAFAC2	0.3149 ± 0.0412	0.3551 ± 0.0360	0.4085 ± 0.0095	0.3881 ± 0.0157	1.3469 ± 0.1304	0.4998 ± 0.0593	0.4948 ± 0.0608	0.4894 ± 0.0207	0.6172 ± 0.0081	1.1392 ± 0.0602
PARAFAC2-I	0.9905 ± 0.0033	1.0075 ± 0.0035	0.9933 ± 0.0024	0.8119 ± 0.0041	1.0203 ± 0.0056	1.0054 ± 0.0014	1.0035 ± 0.0006	1.0087 ± 0.0015	0.8498 ± 0.0019	1.0212 ± 0.0050
PARAFAC2-T	0.9902 ± 0.0025	1.0094 ± 0.0012	0.9974 ± 0.0015	0.8243 ± 0.0026	1.0235 ± 0.0041	1.0097 ± 0.0020	1.0041 ± 0.0004	1.0086 ± 0.0022	0.8558 ± 0.0013	1.0218 ± 0.0032
AO-ADMM	0.5263 ± 0.0038	0.3761 ± 0.0019	0.4645 ± 0.0134	0.9081 ± 0.0268	0.7630 ± 0.0649	0.5944 ± 0.0033	0.4246 ± 0.0052	0.6129 ± 0.0097	1.3270 ± 0.0235	1.0767 ± 0.0071
META-P2 (proposed)	0.2884 ± 0.0287	0.3237 ± 0.0278	0.3849 ± 0.0260	0.2907 ± 0.0042	0.9024 ± 0.0627	0.3896 ± 0.0279	0.3852 ± 0.0295	0.4666 ± 0.0228	0.3890 ± 0.0044	0.9078 ± 0.0335

TABLE II
ANOMALY DETECTION PERFORMANCE OF META-P2 AND BASELINES IN TERMS OF F1 SCORE. BOLD NUMBERS DENOTE THE BEST PERFORMANCE. META-P2 SHOWS THE BEST DETECTION PERFORMANCE AMONG THE METHODS IN MOST OF THE CASES.

Anomaly Detection										
Models	Anomaly ratio $r_a = 0.05$					Anomaly ratio $r_a = 0.1$				
	Nasdaq	S&P 500	Korea stock	NATOPS	Cricket	Nasdaq	S&P 500	Korea stock	NATOPS	Cricket
PARAFAC2	0.0069 ± 0.0002	0.0041 ± 0.0002	0.0012 ± 0.0005	0.0297 ± 0.0001	0.4094 ± 0.0203	0.0145 ± 0.0004	0.0066 ± 0.0004	0.0013 ± 0.0006	0.0648 ± 0.0006	0.4742 ± 0.0137
PARAFAC2-I	0.2400 ± 0.0199	0.2482 ± 0.0719	0.4594 ± 0.0530	0.2365 ± 0.0036	0.0867 ± 0.0150	0.2167 ± 0.0222	0.1985 ± 0.0439	0.3830 ± 0.0532	0.2510 ± 0.0019	0.1299 ± 0.0112
PARAFAC2-T	0.2301 ± 0.0397	0.1920 ± 0.0348	0.4017 ± 0.0613	0.2256 ± 0.0033	0.0758 ± 0.0096	0.2096 ± 0.0233	0.1691 ± 0.0177	0.3255 ± 0.0462	0.2354 ± 0.0031	0.1280 ± 0.0104
AO-ADMM	0.1293 ± 0.0061	0.3591 ± 0.0162	0.5100 ± 0.0118	0.1089 ± 0.0052	0.0123 ± 0.0021	0.1972 ± 0.0066	0.4049 ± 0.0060	0.5400 ± 0.0122	0.1161 ± 0.0032	0.0324 ± 0.0024
META-P2 (proposed)	0.3508 ± 0.0097	0.4656 ± 0.0121	0.5959 ± 0.0165	0.2960 ± 0.0017	0.0752 ± 0.0094	0.3912 ± 0.0092	0.4993 ± 0.0107	0.6282 ± 0.0133	0.3298 ± 0.0009	0.1292 ± 0.0085

TABLE III
MISSING VALUE PREDICTION ERRORS FOR META-P AND THE BASELINES. THE LOWEST ERRORS HIGHLIGHTED IN BOLD. IN MOST OF THE CASES, META-P SHOWS THE HIGHEST PERFORMANCE.

Missing Value Prediction									
Models	Missing ratio $r_m = 0.1$				Missing ratio $r_m = 0.3$				
	Articulary	Finger	Urbansound	PEMS	Articulary	Finger	Urbansound	PEMS	
Tucker	0.7095 ± 0.0951	0.2800 ± 0.0018	0.8310 ± 0.0067	0.2464 ± 0.0009	0.7606 ± 0.0497	0.3506 ± 0.0013	0.8724 ± 0.0063	0.3128 ± 0.0003	
Tucker-I	0.4851 ± 0.0007	0.2835 ± 0.0006	0.8416 ± 0.0068	0.1798 ± 0.0001	0.5348 ± 0.0054	0.3530 ± 0.0014	0.9042 ± 0.0155	0.2481 ± 0.0001	
Tucker-T	0.4869 ± 0.0020	0.2818 ± 0.0010	0.8383 ± 0.0048	0.1798 ± 0.0001	0.5296 ± 0.0018	0.3531 ± 0.0020	0.9011 ± 0.0135	0.2482 ± 0.0001	
PARAFAC	0.3307 ± 0.0436	0.0525 ± 0.0004	0.5068 ± 0.0187	0.1089 ± 0.0003	0.4198 ± 0.0444	0.1490 ± 0.0009	0.5761 ± 0.0091	0.1860 ± 0.0003	
PARAFAC-I	0.1156 ± 0.0027	0.0559 ± 0.0005	0.8614 ± 0.0045	0.0737 ± 0.0002	0.2100 ± 0.0024	0.1839 ± 0.0025	0.9190 ± 0.0019	0.1553 ± 0.0001	
PARAFAC-T	0.1122 ± 0.0015	0.0567 ± 0.0006	0.8537 ± 0.0037	0.0732 ± 0.0001	0.2103 ± 0.0020	0.1837 ± 0.0022	0.9121 ± 0.0032	0.1551 ± 0.0002	
META-P (proposed)	0.0901 ± 0.0018	0.0310 ± 0.0006	0.5130 ± 0.0095	0.0617 ± 0.0002	0.1141 ± 0.0015	0.0509 ± 0.0005	0.6324 ± 0.0095	0.0732 ± 0.0002	

TABLE IV
ANOMALY DETECTION PERFORMANCE OF META-P AND BASELINES IN TERMS OF F1 SCORE. BOLD NUMBERS DENOTE THE HIGHEST SCORE. META-P SHOWS THE BEST DETECTION PERFORMANCE.

Anomaly Detection									
Models	Anomaly ratio $r_a = 0.05$				Anomaly ratio $r_a = 0.1$				
	Articulary	Finger	Urbansound	PEMS	Articulary	Finger	Urbansound	PEMS	
Tucker	0.0636 ± 0.0016	0.0611 ± 0.0008	0.0574 ± 0.0003	0.0557 ± 0.0001	0.1055 ± 0.0025	0.1198 ± 0.0007	0.1034 ± 0.0003	0.1042 ± 0.0001	
Tucker-I	0.0971 ± 0.0015	0.2584 ± 0.0017	0.0762 ± 0.0007	0.6857 ± 0.0009	0.1147 ± 0.0016	0.3311 ± 0.0030	0.1307 ± 0.0006	0.7298 ± 0.0005	
Tucker-T	0.0977 ± 0.0019	0.2581 ± 0.0033	0.0760 ± 0.0006	0.6844 ± 0.0003	0.1156 ± 0.0017	0.3281 ± 0.0012	0.1304 ± 0.0006	0.7289 ± 0.0008	
PARAFAC	0.0585 ± 0.0015	0.0579 ± 0.0008	0.0567 ± 0.0002	0.0553 ± 0.0002	0.1049 ± 0.0017	0.1163 ± 0.0009	0.1028 ± 0.0004	0.1037 ± 0.0001	
PARAFAC-I	0.2435 ± 0.0048	0.4341 ± 0.0028	0.1009 ± 0.0008	0.7134 ± 0.0006	0.2787 ± 0.0029	0.4869 ± 0.0018	0.1611 ± 0.0012	0.7678 ± 0.0004	
PARAFAC-T	0.2431 ± 0.0059	0.4301 ± 0.0027	0.1005 ± 0.0008	0.7124 ± 0.0009	0.2828 ± 0.0029	0.4850 ± 0.0018	0.1614 ± 0.0009	0.7672 ± 0.0005	
META-P (proposed)	0.2465 ± 0.0053	0.4347 ± 0.0028	0.1009 ± 0.0006	0.7127 ± 0.0006	0.2847 ± 0.0037	0.4886 ± 0.0024	0.1622 ± 0.0005	0.7671 ± 0.0009	

using the results of PARAFAC decomposition on source domains. Tucker [29] performs Tucker decomposition on both the source and target domains. Tucker-T applies Tucker decomposition only on the target domain, while Tucker-I initializes the latent factors of the target domain based on the results of the Tucker decomposition from source domains.

3) *Evaluation and Settings*: To evaluate the quality of decomposed results of META-P2, we perform two downstream tasks: missing value prediction and anomaly detection. In the missing value prediction task, we use the normalized error as the evaluation metric; given an original tensor $\mathbf{X}$ and the reconstructed one $\hat{\mathbf{X}}$, the normalized error r_{err} is:

$$r_{\text{err}} = \frac{\sum_{k=1}^K \sum_{(i,j) \in \Omega_k} \|\mathbf{X}_k(i,j) - \hat{\mathbf{X}}_k(i,j)\|_F^2}{\sum_{k=1}^K \sum_{(i,j) \in \Omega_k} \|\mathbf{X}_k(i,j)\|_F^2} \quad (28)$$

where $\mathbf{X}_k$ is the k -th frontal slice, $\hat{\mathbf{X}}_k$ is the reconstructed one of $\mathbf{X}_k$, and Ω_k is the set of test entries of the k -th slice matrix. In the anomaly detection task, we use the F1 score as the evaluation metric.

For each dataset, we set the last domain (in alphabetical

order) as the target, and the rest as source domains. For example, in stock datasets, we align sectors alphabetically and use the last sector, *Utilities* in our case, as the target domain. We randomly split the data of the target domain into training and test missing (anomalous) entries with the missing (anomaly) ratio. For each dataset, we set the target rank R as the minimum mode length of the target tensor, and the maximum number of iterations as 10. We run all experiments ten times with random seeds, and report the average and standard deviation removing the highest and the lowest results.

B. Missing Value Prediction (Q1)

In Table I, we present the missing value prediction errors of META-P2 and other irregular tensor decomposition methods. In Table III, we compare the missing value prediction performance of META-P with that of other regular tensor decomposition methods. We vary the missing ratio $r_m \in \{0.1, 0.3\}$.

PARAFAC2-I in Table I, and PARAFAC-I and Tucker-I in Table III show poor performance, as they rely on naive de-

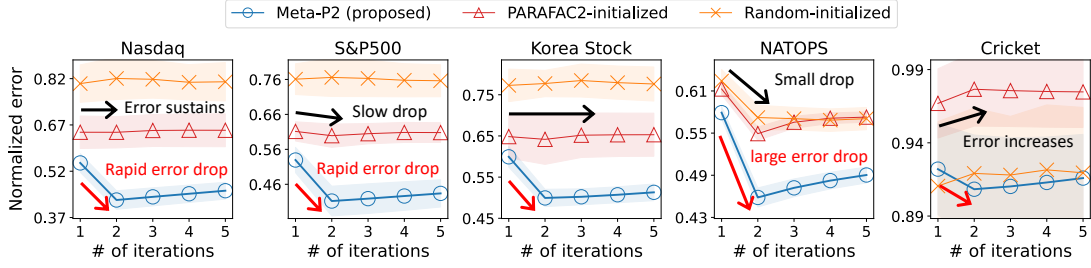


Fig. 3. The missing value prediction performance over the ALS iterations. We compare META-P2 with two other initialization approaches of $\mathbf{V}_T$ for a target domain T . Note that the error of META-P2 drops rapidly, achieving the best prediction performance among the initialization methods. The errors of other baselines drop slowly, or even increase in Cricket.

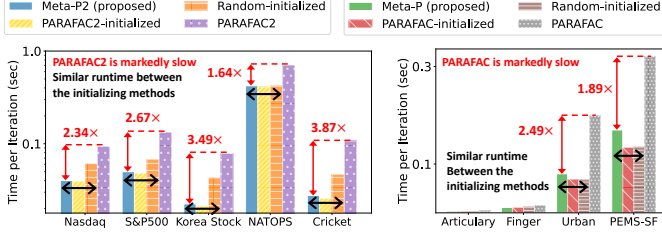


Fig. 4. Running time for each ALS iteration on a target domain. META-P2 and META-P are significantly faster than PARAFAC2 and PARAFAC, respectively, across diverse datasets. This is because PARAFAC2 and PARAFAC require decomposing the tensors in entire domains whenever a new target irregular tensor emerges. The runtimes of META-P2 and META-P remain comparable to other initialization-based methods.

composition from source domains without proper adaptation. This highlights the need to extract easily adaptable information from the sources. PARAFAC2, PARAFAC, and Tucker require re-decomposing all domains for each new target (despite being more costly; see Fig. 4), yet still underperform. These results confirm the effectiveness of focusing on direct adaptation to the target domain. As a result, META-P2 and META-P achieve the lowest errors in both irregular and regular tensor settings.

For Cricket, the performance gain of META-P2 is relatively small. This is due to the high dissimilarity between the target and source domains, which limits the effectiveness of the shared meta factor. Addressing such extreme domain shifts remains an important direction for future work.

C. Anomaly Detection (Q2)

We randomly select anomalous entries in the target tensor with anomaly ratio $r_a \in \{0.05, 0.1\}$. Then we change the values of the entries by dividing them with $\psi = 2$, to make them anomalies. We detect K anomalies by finding top- K entries with the highest error between the anomaly-injected tensor and our reconstructed one.

Tables II and IV present the anomaly detection performance of the methods in terms of F1 score. In Table II, META-P2 shows superior performance compared to other irregular tensor decomposition methods in most cases. In Table IV, META-P consistently achieves the highest F1 scores among the regular tensor decomposition methods. This indicates that META-P2 and META-P successfully discover the factor matrices that contribute to accurately detecting anomalies in both new irregular and regular tensors.

In Cricket of Table II, PARAFAC2 shows lower error than META-P2. This is because Cricket has extremely small

TABLE V
ABLATION STUDY ON MISSING VALUE PREDICTION PROBLEM. THE LOWEST ERRORS ARE IN BOLD. NOTE THAT EACH MODULE OF META-P2 IMPROVES THE PERFORMANCE OF MISSING VALUE PREDICTION.

Missing Value Prediction (missing ratio $r_m = 0.5$)				
Datasets	MTD-imp	MTD-res	MTD-meta	META-P2 (proposed)
Nasdaq	0.9240 $\pm$ 0.0093	0.7079 $\pm$ 0.0433	0.6526 $\pm$ 0.0396	0.5102 $\pm$ 0.0261
SP500	0.8721 $\pm$ 0.0152	0.6671 $\pm$ 0.0302	0.6053 $\pm$ 0.0209	0.4712 $\pm$ 0.0251
Korea stock	0.9100 $\pm$ 0.0274	0.7242 $\pm$ 0.0401	0.6530 $\pm$ 0.0417	0.5595 $\pm$ 0.0172
NATOPS	0.7887 $\pm$ 0.0012	0.6105 $\pm$ 0.0024	0.5728 $\pm$ 0.0055	0.5097 $\pm$ 0.0042
Cricket	1.0119 $\pm$ 0.0045	0.9702 $\pm$ 0.0262	0.9763 $\pm$ 0.0161	0.9297 $\pm$ 0.0205

number of entries in the target domain, with excessively small number of anomalies. Thus, PARAFAC2 easily neglects the anomalous entries while reconstructing the tensor of entire domains, showing higher reconstruction performance on those anomalies. However, PARAFAC2 completely fails to detect anomalies in other datasets, and does not perform well for missing value prediction with markedly slower speed (Fig. 4).

D. Domain Adaptation Speed (Q3)

META-P2 initializes the latent factor matrix $\mathbf{V}_T$ of the target domain T with the meta factor matrix $\mathbf{V}_{\text{meta}}$. To show that $\mathbf{V}_{\text{meta}}$ adapts to the target domain quickly and effectively, we compare the missing value prediction performance of META-P2 with differently initialized $\mathbf{V}_T$. PARAFAC2-initialized method initializes $\mathbf{V}_T$ with the decomposed result of naive PARAFAC2 decomposition on the source domains. Random-initialized method initializes $\mathbf{V}_T$ randomly. We adopt the imputation process of missing entries and the residual connection of META-P2 to those baselines. We report the performance of the methods over the ALS iterations in Fig. 3. We also show the runtime of them per iteration in Fig. 4.

Fig. 3 shows that META-P2 effectively reduces the prediction error with only a few ALS iterations where the runtime per ALS iteration is the same for all initializing methods (Fig. 4). The prediction errors of other methods are generally monotonous and poor compared to META-P2. This shows that initializing $\mathbf{V}_T$ with $\mathbf{V}_{\text{meta}}$ assists in discovering accurate latent factor matrices of the target irregular tensor quickly. Moreover, PARAFAC2-initialized consistently shows poor performance. This implies that simply tuning the decomposed results to a new target domain cannot consider the difference between source and target domains.

E. Ablation Study (Q4)

We provide an ablation study on the missing value prediction with respect to the effectiveness of each module in META-

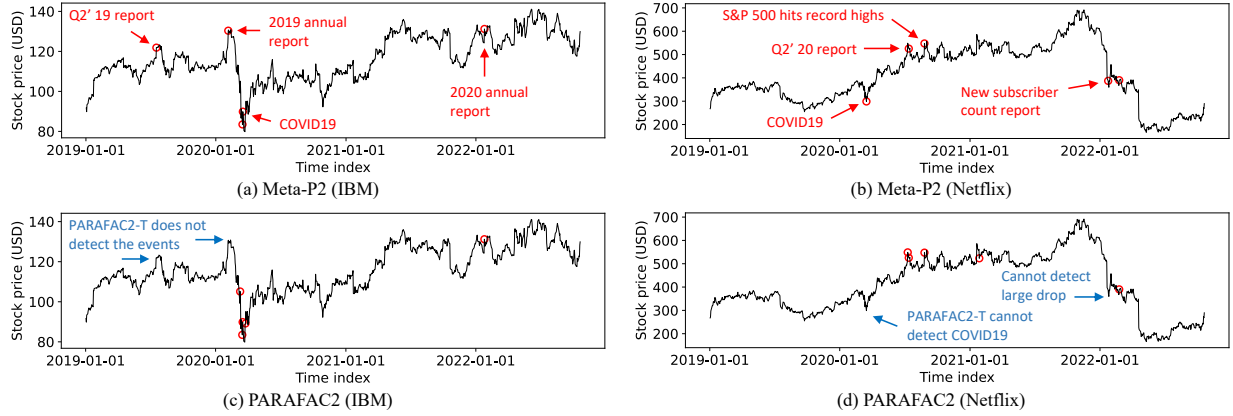


Fig. 5. Top-5 anomaly detection results of META-P2 on (a) IBM and (b) Netflix, and PARAFAC2-T on (c) IBM and (d) Netflix. Note that the detected anomalies (red circles) of META-P2 are closely related to real-world events during the corresponding periods. PARAFAC2-T detect densely distributed abnormal points, leading to the oversight of various real-world events.

P2. META-P2-meta is META-P2 without the meta-learning-based decomposition on the source domains; it performs a naive PARAFAC2 decomposition instead of meta-learning-based decomposition on the source domains, and use the naively decomposed result when adapting to a new target domain. META-P2-imp and META-P2-res are META-P2 without considering the missing values by imputation, and META-P2 without the residual connection of the meta factor, respectively.

Table V shows that each module of META-P2 effectively reduces the prediction error. Specifically, META-P2-meta shows poor performance compared to the proposed META-P2 in every case. This indicates that the meta-learning-based decomposition on source domains effectively enhances the quality of decomposition result of a new target domain.

F. Case Study (Q5)

We showed that META-P2 successfully detects or predicts the abnormal values through Sections V-B and V-C. Our further question is, *how is an anomalous point related to a real-world event?* To answer this, we present discoveries of analyzing historical prices of S&P 500. We compose a target domain by selecting ten largest AI-themed stocks within S&P 500 dataset (Fig. 1), while keeping other settings consistent with those described in Section V-A. We detect five abnormal days by finding days with the top-5 largest errors between the true stock features and the reconstructed ones.

Fig. 5 compares the abnormal days detected by META-P2 and PARAFAC2-T in IBM and Netflix. We observe that the detected days of META-P2 are closely related to real-world events, showing its efficacy in interpreting anomalies. We also note that the detected points of PARAFAC2-T are densely distributed compared to META-P2, failing to detect several real-world events that META-P2 has successfully identified.

Importantly, our method is readily applicable to real-world domains such as healthcare and finance, where data often arrive as domain-partitioned tensors (e.g., by disease type or market sector). In our S&P 500 case study, META-P2 effectively identifies anomalies in a newly introduced domain (AI-themed stocks) without retraining on previous sectors, showcasing its scalability and ease of deployment.

VI. RELATED WORKS

A. Irregular Tensor Decomposition

Most irregular tensor decomposition methods build on the PARAFAC2 model [8], [9], [30]–[32]. Early work such as DPar2 [19], [33] reduces the cost of decomposing dense irregular tensors by reordering and compressing each slice before applying PARAFAC2. However, DPar2 assumes that all tensors arise from a single domain and cannot leverage information from related domains. To scale PARAFAC2 to large, sparse data, SPARTan [20] optimizes the MTTKRP computation, and COPA [21] further incorporates side-information constraints for better accuracy and interpretability. Although these methods achieve impressive efficiency and quality within one domain, they do not address cross-domain scenarios.

Streaming settings are handled by SPADE [34], which fixes non-temporal factors and incrementally updates temporal ones using accelerated MTTKRP, and by Dash [35], which avoids recomputing old data altogether. Both achieve fast online updates but remain confined to the same-domain assumption, limiting their applicability when a new target tensor comes from a different distribution.

Separate from model-agnostic schemes, task-specific approaches such as Atom [36]—which excludes missing entries via an adjusted loss—and LogPar [37]—which models binary observations with a Bernoulli-PU loss—excel at their designated tasks (e.g., missing-value prediction). However, their design hinges on particular downstream objectives and cannot be directly applied to other tasks or domains.

The proposed META-P2 is task-agnostic and domain-adaptive. Unlike prior work that treats each tensor in isolation or within a single domain, META-P2 meta-learns decomposition parameters across multiple source domains and transfers them to an unseen target domain with minimal additional data.

B. Regular Tensor Decomposition

The classical PARAFAC model [10] decomposes a fixed-shape tensor into rank-one factors and has been applied in model compression [38]–[41], anomaly detection [42]–[44], forecasting [45]–[47], time-range query answering [48],

and missing-value imputation [49]. Traditional PARAFAC is simple and interpretable but cannot handle irregularities or domain shifts. To capture nonlinear interactions, neural tensor factorization methods [50]–[52] embed factorization within deep networks. While they offer greater expressiveness, they incur high training costs and require full retraining when adapting to a new domain, making them impractical for rapid cross-domain deployment.

By explicitly modeling domain variation and learning transferable decomposition strategies, META-P2 departs from both classical and neural tensor methods. Rather than optimizing for a single, static domain, our framework adapts to heterogeneous domains, combining the interpretability of PARAFAC2 with the flexibility of meta-learning.

C. Meta-learning

Many meta-learning methods have been proposed to deal with the problem of limited data in many machine learning areas [53], [54]. PMAML [55] is a probabilistic model-agnostic meta-learning method that samples models for a new task from a model distribution. FTML [54] and MOML [56] consider online meta-learning settings where tasks are revealed one after another. However, those previous works are not directly applicable to PARAFAC2 or PARAFAC, as they update model parameters via gradient descent, whereas the decomposition methods update them through CP-ALS.

Previous works have used meta-learning algorithms to solve few-shot learning problems. Chien et al. [57] employed meta-learning for hyperparameter optimization in dialogue system. Gupta et al. [58] used unsupervised meta-learning for reinforcement learning. Lian et al. [59] applied meta-learning to neural architecture search to learn a meta-architecture that is able to adapt to a new task quickly through a few gradient steps. Kim et al. [60] first propose domain adaptation method for decomposing irregular tensors based on meta-learning, which is our preliminary work. In this work, we extend it to support the decomposition of regular tensors.

VII. CONCLUSION

In this work, we have proposed META-P2 and META-P, fast and accurate domain-adaptation methods for decomposing irregular and regular tensors by meta-learning a shared meta factor from multiple source domains and transferring it to an unseen target with only a few ALS updates. Our two-stage workflow—offline meta-training on historical domains followed by lightweight adaptation—enables rapid deployment without re-decomposing past data, with adaptation times comparable to a few ALS iterations. Extensive experiments on real-world datasets demonstrate that our methods achieve state-of-the-art accuracy on missing-value imputation and anomaly detection. Future work will focus on four directions: 1) extending META-P2 to other gradient-based hybrid meta-learning methods, 2) extending the framework to streaming settings, 3) deriving formal convergence bounds and mechanisms to address extreme domain shifts, and 4) pursuing interpretability to link the learned factors to meaningful semantics.

VIII. ACKNOWLEDGMENTS

This work was supported by Institute of Information & communications Technology Planning & Evaluation (IITP) grant funded by the Korea government (MSIT) [No.2022-0-00641, XVoice: Multi-Modal Voice Meta Learning], [No.RS-2025-25442338, AI star Fellowship Support Program (Seoul National University)], [No.RS-2024-00509257, Global AI Frontier Lab], and [No.RS-2021-II211343, Artificial Intelligence Graduate School Program (Seoul National University)]. The Institute of Engineering Research at Seoul National University and the Institute of Computer Technology at Seoul National University provided research facilities for this work. U Kang is the corresponding author.

REFERENCES

- [1] Y. Lin, J. Sun, P. C. Castro, R. B. Konuru, H. Sundaram, and A. Kelliher, “Metafac: community discovery via relational hypergraph factorization,” in *SIGKDD*, 2009.
- [2] S. Spiegel, J. H. Clausen, S. Albayrak, and J. Kunegis, “Link prediction on evolving data using tensor factorization,” in *PAKDD*, 2011.
- [3] J. Jang and U. Kang, “Fast and memory-efficient tucker decomposition for answering diverse time range queries,” in *SIGKDD*, 2021.
- [4] J. Jang, D. Choi, J. Jung, and U. Kang, “Zoom-svd: Fast and memory efficient method for extracting key patterns in an arbitrary time range,” in *CIKM*, 2018.
- [5] R. A. Harshman, “Parafac2: Mathematical and technical notes,” in *UCLA Working Papers in Phonetics*, 1972.
- [6] D. Ahn, S. Kim, and U. Kang, “Accurate online tensor factorization for temporal tensor streams with missing values,” in *CIKM*, 2021.
- [7] P. A. Chew, B. W. Bader, S. Helmreich, A. Abdelali, and S. J. Verzi, “An information-theoretic, vector-space-model approach to cross-language information retrieval,” *Nat. Lang. Eng.*, 2011.
- [8] K. Yin, A. Afshar, J. C. Ho, W. K. Cheung, C. Zhang, and J. Sun, “Logpar: Logistic PARAFAC2 factorization for temporal binary data with missing values,” in *SIGKDD*, 2020.
- [9] E. Pantraki and C. Kotropoulos, “Automatic image tagging and recommendation via PARAFAC2,” in *MLSP*, 2015.
- [10] R. A. Harshman et al., “Foundations of the parafac procedure: Models and conditions for an “explanatory” multi-modal factor analysis,” *UCLA working papers in phonetics*, 1970.
- [11] E. E. Papalexakis, L. Akoglu, and D. Ience, “Do more views of a graph help? community detection and clustering in multi-graphs,” in *FUSION*, 2013.
- [12] G. Tomasi and R. Bro, “Parafac and missing values,” *Chemometrics and Intelligent Laboratory Systems*, 2005.
- [13] A. Streit, G. H. Santos, R. M. Leão, E. d. S. e Silva, D. Menasché, and D. Towsley, “Network anomaly detection based on tensor decomposition,” *Computer Networks*, 2021.
- [14] D. Zachariah, M. Sundin, M. Jansson, and S. Chatterjee, “Alternating least-squares for low-rank matrix reconstruction,” *Signal Process. Lett.*, 2012.
- [15] M. A. Jamal and G. Qi, “Task agnostic meta-learning for few-shot learning,” in *CVPR*, 2019.
- [16] R. Vilalta and Y. Drissi, “A perspective view and survey of meta-learning,” *Artif. Intell. Rev.*, 2002.
- [17] C. Finn, P. Abbeel, and S. Levine, “Model-agnostic meta-learning for fast adaptation of deep networks,” in *ICML*, 2017.
- [18] A. Nichol, “On first-order meta-learning algorithms,” *arXiv:1803.02999*, 2018.
- [19] J. Jang and U. Kang, “Dpar2: Fast and scalable PARAFAC2 decomposition for irregular dense tensors,” in *ICDE*, 2022.
- [20] I. Perros, E. E. Papalexakis, F. Wang, R. W. Vuduc, E. Searles, M. Thompson, and J. Sun, “Spartan: Scalable PARAFAC2 for large & sparse data,” in *SIGKDD*, 2017.
- [21] A. Afshar, I. Perros, E. E. Papalexakis, E. Searles, J. C. Ho, and J. Sun, “COPA: constrained PARAFAC2 for sparse & large datasets,” in *CIKM*, 2018.
- [22] Y. Song, D. Demirdjian, and R. Davis, “Tracking body and hands for gesture recognition: Natops aircraft handling signals database,” in *International Conference on Automatic Face & Gesture Recognition*. IEEE, 2011.

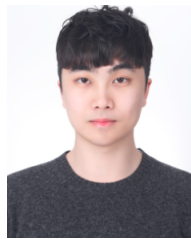
- [23] M. H. Ko, G. West, S. Venkatesh, and M. Kumar, "Online context recognition in multisensor systems using dynamic time warping," in *International Conference on Intelligent Sensors, Sensor Networks and Information Processing*. IEEE, 2005.
- [24] J. Wang, A. Balasubramanian, L. G. M. de la Vega, J. R. Green, A. Samal, and B. Prabhakaran, "Word recognition from continuous articulatory movement time-series data using symbolic representations," in *Proceedings of the Fourth Workshop on Speech and Language Processing for Assistive Technologies*, 2013.
- [25] B. Blankertz, G. Curio, and K.-R. Müller, "Classifying single trial eeg: Towards brain computer interfacing," *NeurIPS*, 2001.
- [26] J. Salamon, C. Jacoby, and J. P. Bello, "A dataset and taxonomy for urban sound research," in *international conference on Multimedia*, 2014.
- [27] M. Cuturi, "Fast global alignment kernels," in *ICML*, 2011.
- [28] M. Roald, C. Schenker, V. D. Calhoun, T. Adali, R. Bro, J. E. Cohen, and E. Acar, "An ao-admm approach to constraining parafac2 on all modes," *SIAM Journal on Mathematics of Data Science*, 2022.
- [29] F. L. Hitchcock, "The expression of a tensor or a polyadic as a sum of products," *Journal of Mathematics and Physics*, 1927.
- [30] P. A. Chew, B. W. Bader, T. G. Kolda, and A. Abdelali, "Cross-language information retrieval using PARAFAC2," in *SIGKDD*, 2007.
- [31] Y. Panagakis and C. Kotropoulos, "Automatic music tagging via PARAFAC2," in *ICASSP*, 2011.
- [32] B. M. Wise, N. B. Gallagher, and E. B. Martin, "Application of parafac2 to fault detection and diagnosis in semiconductor etch," *Journal of Chemometrics*, 2001.
- [33] J. Jang and U. Kang, "Static and streaming tucker decomposition for dense tensors," *TKDD*, 2023.
- [34] E. Gujral, G. Theodorou, and E. E. Papalexakis, "SPADE: streaming PARAFAC2 decomposition for large datasets," in *SDM*, 2020.
- [35] J. Jang, J. Lee, Y. Park, and U. Kang, "Fast and accurate dual-way streaming PARAFAC2 for irregular tensors - algorithm and application," in *SIGKDD*, 2023.
- [36] J.-G. Jang, J. Lee, J. Park, and U. Kang, "Accurate parafac2 decomposition for temporal irregular tensors with missing values," in *Big Data*, 2022.
- [37] K. Yin, A. Afshar, J. C. Ho, W. K. Cheung, C. Zhang, and J. Sun, "Logpar: Logistic parafac2 factorization for temporal binary data with missing values," in *SIGKDD*, 2020.
- [38] D. Chen and J. Zhou, "Lightmobilebert: A secondary lightweight model based on mobilebert," *J. Intell. Fuzzy Syst.*, 2023.
- [39] A. Tjandra, S. Sakti, and S. Nakamura, "Tensor decomposition for compressing recurrent neural network," in *IJCNN*, 2018.
- [40] A. H. Phan, K. Sobolev, K. Sozykin, D. Ermilov, J. Gusak, P. Tichavský, V. Glukhov, I. V. Oseledets, and A. Cichocki, "Stable low-rank tensor decomposition for compression of convolutional neural network," in *ECCV*, 2020.
- [41] M. Alnemari and N. Bagherzadeh, "Ultimate compression: Joint method of quantization and tensor decomposition for compact models on the edge," *Applied Sciences*, 2024.
- [42] H. Fanaee-T and J. Gama, "Tensor-based anomaly detection: An interdisciplinary survey," *Knowl. Based Syst.*, 2016.
- [43] L. Tisljaric, S. Fernandes, T. Caric, and J. Gama, "Spatiotemporal traffic anomaly detection on urban road network using tensor decomposition method," in *Discovery Science*, 2020.
- [44] M. McNeil, C. Mattsson, F. W. Takes, and P. Bogdanov, "Cadence: Community-aware detection of dynamic network states," in *SDM*, 2023.
- [45] G. Cardillo, P. Giordani, S. Levantesi, A. Nigri, and A. Spelta, "Mortality forecasting using the four-way candeomp/parafac decomposition," *Scandinavian Actuarial Journal*, 2023.
- [46] B. Ermiş, E. Acar, and A. T. Cemgil, "Link prediction in heterogeneous data via generalized coupled tensor factorization," *Data Min. Knowl. Discov.*, 2015.
- [47] J. Gardner, D. Koutra, J. Mroueh, V. Pang, A. Farahi, S. Krassenstein, and J. Webb, "Driving with data: Modeling and forecasting vehicle fleet maintenance in detroit," *arXiv preprint*, 2017.
- [48] D. Ahn, J. Jang, and U. Kang, "Time-aware tensor decomposition for sparse tensors," *Mach. Learn.*, 2022.
- [49] M.-Y. Dong, H.-L. Wu, T. Wang, K. Huang, H. Ren, and R.-Q. Yu, "Parafacm: A second-order calibration algorithm for handling data with missing values," *Chemometrics and Intelligent Systems*, 2024.
- [50] X. Wu, B. Shi, Y. Dong, C. Huang, and N. V. Chawla, "Neural tensor factorization for temporal interaction learning," in *WSDM*, 2019.
- [51] H. Chen and J. Li, "Neural tensor model for learning multi-aspect factors in recommender systems," in *IJCAI*, 2020.
- [52] B. Liu, L. He, Y. Li, S. Zhe, and Z. Xu, "Neuralcp: Bayesian multiway data analysis with neural tensor decomposition," *Cogn. Comput.*, 2018.
- [53] T. M. Hospedales, A. Antoniou, P. Micaelli, and A. J. Storkey, "Meta-learning in neural networks: A survey," *IEEE Trans. Pattern Anal. Mach. Intell.*, 2022.
- [54] C. Finn, A. Rajeswaran, S. Kakade, and S. Levine, "Online meta-learning," in *International conference on machine learning*. PMLR, 2019, pp. 1920–1930.
- [55] C. Finn, K. Xu, and S. Levine, "Probabilistic model-agnostic meta-learning," in *NeurIPS*, 2018.
- [56] D. A. E. Acar, R. Zhu, and V. Saligrama, "Memory efficient online meta learning," in *ICML*, 2021.
- [57] J. Chien and W. X. Lieow, "Meta learning for hyperparameter optimization in dialogue system," in *Interspeech*, 2019.
- [58] A. Gupta, B. Eysenbach, C. Finn, and S. Levine, "Unsupervised meta-learning for reinforcement learning," *CoRR*, 2018.
- [59] D. Lian, Y. Zheng, Y. Xu, Y. Lu, L. Lin, P. Zhao, J. Huang, and S. Gao, "Towards fast adaptation of neural architectures with meta learning," in *ICLR*, 2020.
- [60] J. Kim, K. H. Park, J.-G. Jang, and U. Kang, "Fast and accurate domain adaptation for irregular tensor decomposition," in *SIGKDD*, 2024.



Junghun Kim is a Ph.D candidate in Interdisciplinary Program in Artificial Intelligence at Seoul National University, advised by Prof. U Kang in Data Mining Laboratory. His research interests include deep learning and graph mining.



Ka Hyun Park is a Ph.D candidate in the Department of Computer Science and Engineering of Seoul National University, advised by Prof. U Kang in Data Mining Laboratory. Her research interests include deep learning and graph mining.



Jun-Gi Jang is an Assistant Professor in the Department of Electrical Engineering and Computer Science at DGIST. He was a postdoctoral research fellow at University of Illinois Urbana-Champaign, hosted by Prof. Hanghang Tong. He received Ph.D in Computer Science and Engineering at Seoul National University, where he was advised by Prof. U Kang. He received B.S. in Mechanical & Aerospace Engineering at Seoul National University. His research interests include tensor mining and time series analysis.



U Kang is a Professor in the Department of Computer Science and Engineering of Seoul National University, where he also serves as the head Professor of the Interdisciplinary Program in Artificial Intelligence. He received Ph.D. in Computer Science at Carnegie Mellon University, after receiving B.S. in Computer Science and Engineering at Seoul National University. He won 2013 SIGKDD Doctoral Dissertation Award, 2013 New Faculty Award from Microsoft Research Asia, 2016 Korean Young Information Scientist Award, and six 'best paper' awards including 2018 ICDM 10-years best paper award and 2021 KDD best research paper award. He has published over 100 refereed articles in major data mining and machine learning venues. His research interests include data mining and machine learning.